COC2070 – Digital Logic and System Design

Prof. M. M. Sufyan Beg
Department of Computer Engineering
Z. H. College of Engineering & Technology
Aligarh Muslim University, India

Lecture-1: Number System

- Logic – Reasoning
- It all starts with the **<u>Number System:</u>**
- $4038 = 4\times10^3 + 0\times10^2 + 3\times10^1 + 8\times10^0$
- $(317.63)_{10} = 3\times10^2 + 1\times10^1 + 7\times10^0 + 6\times10^{-1} + 3\times10^{-2}$
- This is how we do it in base 10 or the decimal numbers
- We can extend the same method to other bases
- $(426.041)_7 = 4\times7^2 + 2\times7^1 + 6\times7^0 + 4\times7^{-2} + 1\times7^{-3} = (216.0845)_{10}$
- $(8B.A3)_{12} = 8\times12^1 + 11\times12^0 + 10\times12^{-1} + 3\times12^{-2} = (107.854)_{10}$
- A number in base *r* will have digits from 0 to (*r*-1)
- For bases beyond 10, we will have A for 10, B for 11, C for 12, D for 13, E for 14, F for 15, and so on.

Lecture-1 (continued)

- Base 10 numbers are Decimal, Base 16 are Hexadecimal, Base 8 are Octal and Base 2 are Binary Numbers
- While it is easier to convert a number in base *r* to a decimal number, the reverse may not be that easy
- For converting a decimal number to a number in another base *r*, we will have to work on the whole part and the fraction part separately
- Also, for converting a number from base *m* to a base *n*, we will have to first convert the base-*m* number to decimal and then convert this decimal number to base-*n*
- Direct conversions may only be done between special cases like Binary-Octal, Binary-Hexadecimal, etc.

Converting $(3812.006)_{10}$ to base-11

- We shall convert the whole part first, as follows:
- We shall divide the whole part 3812 by 11 successively and keep noting the quotient and remainders

Base to be converted into	Successive Quotient	Successive Remainder
11	3812	
	346	6
	31	5
	2	9
	0	2

- Reading the Remainders from **<u>bottom-to-up</u>**, we get $(3812)_{10} = (2956)_{11}$

Converting $(3812.006)_{10}$ to base-11 (contd.)

- We shall now convert the fraction part $(0.006)_{10}$ by successively multiplying the fraction of each step by 11 as follows:
- $11 \times 0.006 = 0.066$
- $11 \times 0.066 = 0.726$
- $11 \times 0.726 = 7.986$
- $11 \times 0.986 = A.846$
- $11 \times 0.846 = 9.306$
- Now reading the whole part of the products from **<u>top-to-bottom</u>**, we get $(0.006)_{10} = (0.007A9)_{11}$
- Just combine the converted whole part and the fraction part together to get $(3812.006)_{10} = (2956.007A9)_{11}$
- We can now cross check to see that:
- $(2956.007A9)_{11} = 2\times11^3 + 9\times11^2 + 5\times11^1 + 6\times11^0 + 0\times11^{-1} + 0\times11^{-2} + 7\times11^{-3} + A\times11^{-4} + 9\times11^{-5} = (3812.00599)_{10}$, which is very close to $(3812.006)_{10}$

Converting $(610.32)_{10}$ to Binary

- We shall convert the whole part first, as follows:
- We shall divide the whole part 610 by 2 successively and keep noting the quotient and remainders

Base to be converted into	Successive Quotient	Successive Remainder
2	610	
	305	0
	152	1
	76	0
	38	0
	19	0
	9	1
	4	1
	2	0
	1	0
	0	1

- Reading the Remainders from **<u>bottom-to-up</u>**, we get $(610)_{10} = (1001100010)_2$

Converting $(610.32)_{10}$ to Binary (contd.)

- We shall now convert the fraction part $(0.32)_{10}$ by successively multiplying the fraction of each step by 2 as follows:
- $2 \times 0.32 = 0.64$
- $2 \times 0.64 = 1.28$
- $2 \times 0.28 = 0.56$
- $2 \times 0.56 = 1.12$
- $2 \times 0.12 = 0.24$
- $2 \times 0.24 = 0.48$
- $2 \times 0.48 = 0.96$
- $2 \times 0.96 = 1.92$
- $2 \times 0.92 = 1.84$
- $2 \times 0.84 = 1.68$
- $2 \times 0.68 = 1.36$
- $2 \times 0.36 = 0.72$
- $2 \times 0.72 = 1.44$

Converting $(610.32)_{10}$ to Binary (contd.)

- Now reading the whole part of the products from **<u>top-to-bottom</u>**, we get $(0.32)_{10} = (0.0101000111101)_2$
- Just combine the converted whole part and the fraction part together to get $(610.32)_{10} = (1001100010.0101000111101)_2$
- We can now cross check to see that:

$(1001100010.0101000111101)_2$

$= 1\times2^{9} + 1\times2^{6} + 1\times2^{5} + 1\times2^{1} + 1\times2^{-2} + 1\times2^{-4} + 1\times2^{-8} + 1\times2^{-9} + 1\times2^{-10}$

$+ 1\times2^{-11} + 1\times2^{-13}$

$= (610.3199)_{10}$, which is very close to $(610.32)_{10}$

Number System (contd.)

- BInary digiT is called a BIT
- 8-bits put together is called a Byte (e.g. 10110011)
- <u>Direct conversion from Octal to Binary</u>:
- Each Octal digit will be converted to three bits
- e.g. $(437.06)_8 = (100\ 011\ 111.000\ 110)_2$
- <u>Direct conversion from Binary to Octal:</u>
- For the whole part, combine three bits into an Octal digit starting from the decimal and moving leftwards, append zeros at the leftmost part, if required to complete the count of 3 bits
- For the fraction part, combine three bits into an Octal digit starting from the decimal and moving rightwards, append zeros at the rightmost part, if required to complete the count of 3 bits
- e.g. $(011100111.1101)_2 = (\underline{011}\ \underline{100}\ \underline{111}.\underline{110}\ \underline{100})_2 = (347.64)_8$

Direct conversion between Hexadecimal and Binary

- <u>Direct conversion from Hexadecimal to Binary</u>:
- Each Hexadecimal digit will be converted to four bits
- e.g. $(8AC6.10F)_{16} = (1000\ 1010\ 1100\ 0110.0001\ 0000\ 1111)_2$
- <u>Direct conversion from Binary to Hexadecimal:</u>
- For the whole part, combine four bits into a Hexadecimal digit starting from the decimal and moving leftwards, append zeros at the leftmost part, if required to complete the count of 4 bits
- For the fraction part, combine three bits into a Hexadecimal digit starting from the decimal and moving rightwards, append zeros at the rightmost part, if required to complete the count of 4 bits
- e.g. $(1011100111.11011)_2 = (\underline{0010}\ \underline{1110}\ \underline{0111}.\underline{1101}\ \underline{1000})_2 = (2E7.D8)_{16}$
- Can there be more such direct conversions?
- Yes, between Bases 2 and 4, 4 and 16, 3 and 9, etc. $[r_1=(r_2)^n]$

Binary Arithmetic

- <u>Counting in Binary:</u>
- 0, 1, 10, 11, 100, 101, 110, 111, 1000, 1001, 1010, 1011, …
- <u>Addition of two Binary Numbers</u>:
- 0+0=0, 0+1=1+0=1, 1+1=10, 1+1+1=11

Carry			**1**	**1**	**1**	**1**	**1**	**0**		**0**	**0**	
Augend			1	1	1	0	1	1	.	0	0	1
Addend		+	0	0	1	1	0	1	.	1	1	0
Sum		1	0	0	1	0	0	0	.	1	1	1

- <u>Subtraction of two Binary Numbers:</u>

Minuend			1	1	1	0	1	1	.	0	0	1
Subtrahend		-	0	1	1	1	0	1	.	1	1	0
Difference			0	1	1	1	0	1	.	0	1	1

Binary Multiplications

$(26)_{10}$	**Multiplicand**						**1**	**1**	**0**	**1**	**0**
$(13)_{10}$	**Multiplier**					**×**		**1**	**1**	**0**	**1**
							1	1	0	1	0
						0	0	0	0	0	X
					1	1	0	1	0	X	X
				1	1	0	1	0	X	X	X
$(338)_{10}$	**Product**		**1**	**0**	**1**	**0**	**1**	**0**	**0**	**1**	**0**

Complements

- Complements are used for simplifying the subtraction operation and for logical manipulations
- There are 2 types of complements for each base-*r* system:

  (1) the *r*'s complement, and (2) the (*r*-1)'s complement
- *r* 's complement: Given a positive number *N* in base *r* with the integer part of *n* digits, the *r*'s complement of *N* is defined as

  $r^n - N$ for $N \neq 0$ and 0 for $N = 0$
- The 10's complement of $(2186.3304)_{10}$ is 10^4-2186.3304 = $(7813.6696)_{10}$
- The 7's complement of $(601.312)_7$ is 7^3 - $(601.312)_7$ = $(65.355)_7$

Base 7	**:**		**1**	**0**	**0**	**0**	**.**	**0**	**0**	**0**
Base 7	:	-		6	0	1	.	3	1	2
Base 7	:			0	6	5	.	3	5	5

r's Complement (contd.)

- The 14's complement of $(0.3A0D)_{14}$ is $E^0 - (0.3A0D)_{14} = (0.A3D1)_{14}$

Base 14	**:**		**1**	**.**	**0**	**0**	**0**	**0**
Base 14	:	-	0	.	3	A	0	D
Base 14	:		0	.	A	3	D	1

- The 2's complement of $(101100101)_2$ is $2^9 - (101100101)_2 = (010011011)_2$

Base 2	**:**		**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
Base 2	:	-		1	0	1	1	0	0	1	0	1
Base 2	:			0	1	0	0	1	1	0	1	1

- The 2's complement of $(11001.0110)_2$ is $2^5 - (11001.0110)_2 = (10110.1010)_2$

Base 2	**:**		**1**	**0**	**0**	**0**	**0**	**0**	**.**	**0**	**0**	**0**	**0**
Base 2	:	-		1	1	0	0	1	.	0	1	1	0
Base 2	:			0	0	1	1	0	.	1	0	1	0

(r-1)'s Complement

- Given a positive number N in base r with the integer part of n digits and the fraction part of m digits, the (r-1)'s complement of N is defined as $r^n - r^{-m} - N$.
- The (10-1)'s complement of $(384.7026)_{10}$ is $10^3 - 10^{-4} - 384.7026 = 999.9999 - 384.7026 = 615.2973$
- The (6-1)'s complement of $(15032)_6$ is $6^5 - 1 - (15032)_6 = (55555)_6 - (15032)_6 = (40523)_6$
- The (11-1)'s complement of $(0.1A307)_{11}$ is $11^0 - 11^{-5} - (0.1A307)_{11} = (0.AAAAA)_{11} - (0.1A307)_{11} = (0.907A3)_{11}$
- The 1's complement of $(11001011101)_2$ is $2^{11} - 2^0 - (11001011101)_2 = (11111111111)_2 - (11001011101)_2 = (00110100010)_2$
- The 1's complement of $(0.0110110)_2$ is $2^0 - 2^{-7} - (0.0110110)_2 = (0.1111111)_2 - (0.0110110)_2 = (0.1001001)_2$
- The 1's complement of $(10111.1101)_2$ is $2^5 - 2^{-4} - (10111.1101)_2 = (11111.1111)_2 - (10111.1101)_2 = (01000.0010)_2$

Shortcut for obtaining 1's and 2's Complements in binary

- For getting 1's complement: simply change every 1 to 0 and every 0 to 1
- For getting 2's complement, there are 2 methods
- Method 1: Get 1's complement first and then add 1 to the least significant bit (LSB)
- e.g. For 2's complement of (11001.0010), the 1's compliment is (00110.1101) and adding 1 to the LSB, we get the 2's compliment as (00110.1110)

	0	**0**	**1**	**1**	**0**	**.**	**1**	**1**	**0**	**1**
+										1
	0	0	1	1	0	.	1	1	1	0

- Method 2: Start from the rightmost bit and keep copying until we come across the first 1, flip all the bits thereafter.

COC2070 – Digital Logic and System Design

Prof. M. M. Sufyan Beg
Department of Computer Engineering
Z. H. College of Engineering & Technology
Aligarh Muslim University, India

Subtraction by *r* ’s Complement

- Add the Minuend *M* to the *r* ’s complement of subtrahend *N*
- If an end carry occurs, discard it
- If an end carry does not occur, take the *r* ’s complement and place a negative sign
- **<u>Example (1)</u>:** Find 82116.2201 – 32114.331
- 10’s complement of 32114.3310 is 10^5 - 32114.3310 = 67885.6690
- 82116.2201 + 67885.6690 = 1 50001.8891, where the leftmost 1 is a carry and hence needs to be discarded, 50001.8891 is final answer
- **<u>Example (2)</u>:** Find 3078.36 – 9146.201
- 10’s complement of 9146.201 is 10^4 - 9146.201 = 853.799
- 3078.36 + 853.799 = 3932.159 with no end carry. So taking the 10’s complement of 3932.159 as 10^4 - 3932.159 = 6067.841
- Placing a negative sign, we get - 6067.841 as the final answer

Subtraction by 2 ’s Complement

- **<u>Example (1)</u>:** Find 11010.011 – 10111.101
- 2’s complement of 10111.101 is 01000.011
- 11010.011 + 01000.011 = 1 00010.110, where the leftmost 1 is a carry and hence needs to be discarded, 00010.110 is final answer

Direct Subtraction										
		1	1	0	1	0	.	0	1	1
	-	1	0	1	1	1	.	1	0	1
		0	0	0	1	0	.	1	1	0

Check										
		0	0	0	1	0	.	1	1	0
	+	1	0	1	1	1	.	1	0	1
		1	1	0	1	0	.	0	1	1

- **<u>Example (2)</u>:** Find 10111.011 – 11000.11
- 2’s complement of 11000.11 is 00111.01
- 10111.011 + 00111.01 = 11110.101 with no end carry, So taking the 2’s complement of 11110.101, we get 00001.011
- Placing a negative sign, we get - 00001.011 as the final answer

Direct Subtraction										
		1	1	0	0	0	.	1	1	
	-	1	0	1	1	1	.	0	1	1
		0	0	0	0	1	.	0	1	1

Check										
		0	0	0	0	1	.	0	1	1
	+	1	0	1	1	1	.	0	1	1
		1	1	0	0	0	.	1	1	0

Subtraction by (r-1) ’s Complement

- Add the Minuend M to the (r-1)’s complement of subtrahend N
- If an end carry occurs, add 1 to the least significant digit
- If an end carry does not occur, take the (r-1)’s complement and place a negative sign
- **<u>Example (1)</u>:** Find 82116.2201 – 32114.331
- (10-1)’s complement of 32114.3310 is 99999.9999 - 32114.3310 = 67885.6689
- 82116.2201 + 67885.6689 = 1 50001.8890, where the leftmost 1 is a carry and hence needs to be added to the least significant digit
- to give 50001.8890 + 0.0001 = 50001.8891 as the final answer
- **<u>Example (2)</u>:** Find 3078.36 – 9146.201
- (10-1)’s complement of 9146.201 is 9999.999 - 9146.201 =0853.798
- 3078.36 + 0853.798 = 3932.158 with no end carry. So taking the (10-1)’s complement of 3932.158 as 9999.999 - 3932.158=6067.841
- Placing a negative sign, we get - 6067.841 as the final answer

Subtraction by 1's Complement

- **<u>Example (1)</u>:** Find 11010.011 – 10111.101
- 1's complement of 10111.101 is 01000.010
- 11010.011 + 01000.010 = 1 00010.101, where the leftmost 1 is a carry and hence needs to be added to LSB to give 00010.110

Direct Subtraction										
		1	1	0	1	0	.	0	1	1
	-	1	0	1	1	1	.	1	0	1
		0	0	0	1	0	.	1	1	0

Check										
		0	0	0	1	0	.	1	1	0
	+	1	0	1	1	1	.	1	0	1
		1	1	0	1	0	.	0	1	1

- **<u>Example (2)</u>:** Find 10111.011 – 11000.11
- 1's complement of 11000.110 is 00111.001
- 10111.011 + 00111.001 = 11110.100 with no end carry, So taking the 1's complement of 11110.100, we get 00001.011
- Placing a negative sign, we get - 00001.011 as the final answer

Direct Subtraction										
		1	1	0	0	0	.	1	1	
	-	1	0	1	1	1	.	0	1	1
		0	0	0	0	1	.	0	1	1

Check										
		0	0	0	0	1	.	0	1	1
	+	1	0	1	1	1	.	0	1	1
		1	1	0	0	0	.	1	1	0

Advantage of Subtraction by Complement Method

- Subtraction can be done by addition only, provided the complement is known, which is easy in binary number system
  - Saves hardware of subtractor
- Need not have to worry that Minuend must be bigger than the Subtrahend
- Signed Number Representation:

Signed Decimal Number	**Binary**		
	Signed Magnitude Form	**1's Complement Form**	**2's Complement Form**
+82	0 1010010	0 1010010	0 1010010
-82	1 1010010	1 0101101	1 0101110
+127	0 1111111	0 1111111	0 1111111
-127	1 1111111	1 0000000	1 0000001

Comparison between 1's and 2's Complements

- It is easier to find 1's Complement
- Additional step of adding 1 at the LSB for finding 2's Complement
- End around carry, if any, is just to be discarded in the 2's complement method of subtraction
- End around carry to be added to the LSB of the result in the 1's complement method of subtraction
- Confusion of a negative zero in 1's complement method of subtraction while subtracting two equal binary numbers
- e.g. 1011-1011:
- 2's complement method: 1011 + 0101 = 1 0000, discard end carry to get 0000 as the correct answer
- 1's complement method: 1011 + 0100 = 1111, no end carry, so take 1's complement and place a negative sign to get -0000
- So, in the case of the 1's complement method, Negative Zero???

COC2070 – Digital Logic and System Design

Prof. M. M. Sufyan Beg
Department of Computer Engineering
Z. H. College of Engineering & Technology
Aligarh Muslim University, India

Binary Codes

- Binary Coded Decimal (BCD)
  - For 10, binary is 1010, but BCD is 0001 0000
  - For 92, the binary is 1011100, but the BCD is 1001 0010
- Excess-3 Code
  - For 5, Excess-3 Code is 1000
  - For 9, Excess-3 Code is 1100
  - It is like, “I AM GOOD” is coded as “L DP JRRG”

Decimal digit	(BCD) 8421	Excess-3	84-2-1	2421	(Biquinary) 5043210
0	0000	0011	0000	0000	0100001
1	0001	0100	0111	0001	0100010
2	0010	0101	0110	0010	0100100
3	0011	0110	0101	0011	0101000
4	0100	0111	0100	0100	0110000
5	0101	1000	1011	1011	1000001
6	0110	1001	1010	1100	1000010
7	0111	1010	1001	1101	1000100
8	1000	1011	1000	1110	1001000
9	1001	1100	1111	1111	1010000

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Reflected Code/Gray Code

Reflected code	Decimal equivalent
0000	0
0001	1
0011	2
0010	3
0110	4
0111	5
0101	6
0100	7
1100	8
1101	9
1111	10
1110	11
1010	12
1011	13
1001	14
1000	15

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Error Detection Codes

- Parity bit: Single error detection but no correction

(a) Message	P (odd)	(b) Message	P (even)
0000	1	0000	0
0001	0	0001	1
0010	0	0010	1
0011	1	0011	0
0100	0	0100	1
0101	1	0101	0
0110	1	0110	0
0111	0	0111	1
1000	0	1000	1
1001	1	1001	0
1010	1	1010	0
1011	0	1011	1
1100	1	1100	0
1101	0	1101	1
1110	0	1110	1
1111	1	1111	0

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Alpha-Numeric Codes

Character	6-Bit internal code	7-Bit ASCII code	8-Bit EBCDIC code	12-Bit card code
A	010 001	100 0001	1100 0001	12,1
B	010 010	100 0010	1100 0010	12,2
C	010 011	100 0011	1100 0011	12,3
D	010 100	100 0100	1100 0100	12,4
E	010 101	100 0101	1100 0101	12,5
F	010 110	100 0110	1100 0110	12,6
G	010 111	100 0111	1100 0111	12,7
H	011 000	100 1000	1100 1000	12,8
I	011 001	100 1001	1100 1001	12,9
J	100 001	100 1010	1101 0001	11,1
K	100 010	100 1011	1101 0010	11,2
L	100 011	100 1100	1101 0011	11,3
M	100 100	100 1101	1101 0100	11,4
N	100 101	100 1110	1101 0101	11,5
O	100 110	100 1111	1101 0110	11,6
P	100 111	101 0000	1101 0111	11,7
Q	101 000	101 0001	1101 1000	11,8
R	101 001	101 0010	1101 1001	11,9
S	110 010	101 0011	1110 0010	0,2
T	110 011	101 0100	1110 0011	0,3
U	110 100	101 0101	1110 0100	0,4
V	110 101	101 0110	1110 0101	0,5
W	110 110	101 0111	1110 0110	0,6
X	110 111	101 1000	1110 0111	0,7
Y	111 000	101 1001	1110 1000	0,8
Z	111 001	101 1010	1110 1001	0,9
0	000 000	011 0000	1111 0000	0
1	000 001	011 0001	1111 0001	1
2	000 010	011 0010	1111 0010	2
3	000 011	011 0011	1111 0011	3
4	000 100	011 0100	1111 0100	4
5	000 101	011 0101	1111 0101	5
6	000 110	011 0110	1111 0110	6
7	000 111	011 0111	1111 0111	7
8	001 000	011 1000	1111 1000	8
9	001 001	011 1001	1111 1001	9
blank	110 000	010 0000	0100 0000	no punch
.	011 011	010 1110	0100 1011	12,8,3
(	111 100	010 1000	0100 1101	12,8,5
+	010 000	010 1011	0100 1110	12,8,6
$	101 011	010 0100	0101 1011	11,8,3
*	101 100	010 1010	0101 1100	11,8,4
)	011 100	010 1001	0101 1101	11,8,5
–	100 000	010 1101	0110 0000	11
/	110 001	010 1111	0110 0001	0,1
,	111 011	010 1100	0110 1011	0,8,3
=	001 011	011 1101	0111 1110	8,6

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Logic Operations

Binary logic consists of binary variables and logical operations. The variables are designated by letters of the alphabet such as A, B, C, x, y, z, etc., with each variable having two and only two distinct possible values: 1 and 0. There are three basic logical operations: AND, OR, and NOT.

1. AND: This operation is represented by a dot or by the absence of an operator. For example, $x \cdot y = z$ or $xy = z$ is read "x AND y is equal to z." The logical operation AND is interpreted to mean that $z = 1$ if and only if $x = 1$ *and* $y = 1$; otherwise $z = 0$. (Remember that x, y, and z are binary variables and can be equal either to 1 or 0, and nothing else.)

2. OR: This operation is represented by a plus sign. For example, $x + y = z$ is read "x OR y is equal to z," meaning that $z = 1$ if $x = 1$ *or* if $y = 1$ *or* if both $x = 1$ and $y = 1$. If both $x = 0$ and $y = 0$, then $z = 0$.

3. NOT: This operation is represented by a prime (sometimes by a bar). For example, $x' = z$ (or $\bar{x} = z$) is read "x not is equal to z," meaining that z is what x is not. In other words, if $x = 1$, then $z = 0$ but if $x = 0$, then $z = 1$.

Reference: M. Morris Mano, "Digital Logic and Computer Design", PHI

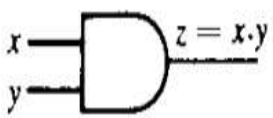
Truth Table of Logic Operations

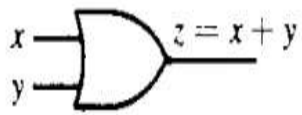
- Truth Table specifies the output for all the possible combinations of inputs

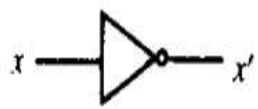
AND		
x	***y***	***x.y***
0	0	0
0	1	0
1	0	0
1	1	1

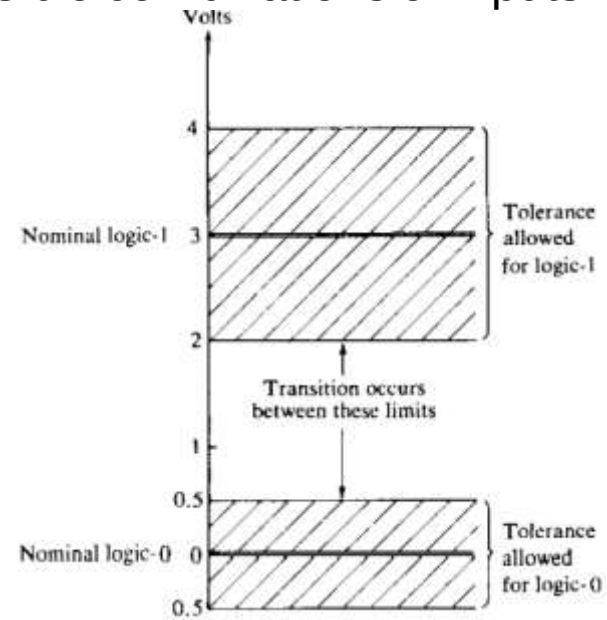
OR		
x	***y***	***x+y***
0	0	0
0	1	1
1	0	1
1	1	1

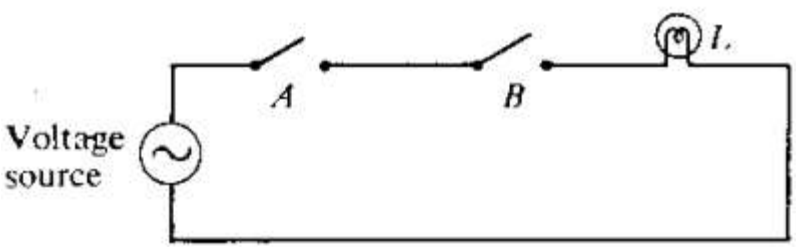
NOT	
x	***x'***
0	1
1	0



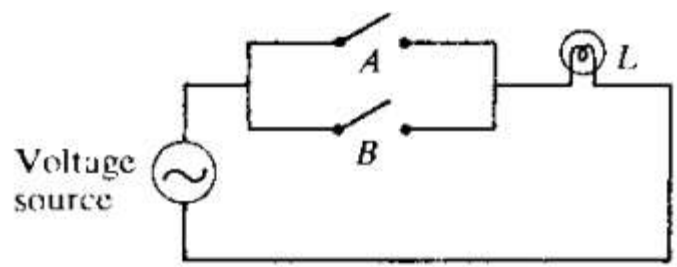








(a) Switches in series – logic AND



(b) Switches in parallel – logic OR

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

COC2070 – Digital Logic and System Design

Prof. M. M. Sufyan Beg
Department of Computer Engineering
Z. H. College of Engineering & Technology
Aligarh Muslim University, India

Boolean Algebra (Huntington Postulates)

Boolean algebra is an algebraic structure defined on a set of elements B together with two binary operators $+$ and $\cdot$ provided the following (Huntington) postulates are satisfied:

1. (a) Closure with respect to the operator $+$.
   (b) Closure with respect to the operator $\cdot$.
2. (a) An identity element with respect to $+$, designated by 0: $x + 0 = 0 + x = x$.
   (b) An identity element with respect to $\cdot$, designated by 1: $x \cdot 1 = 1 \cdot x = x$.
3. (a) Commutative with respect to $+$: $x + y = y + x$.
   (b) Commutative with respect to $\cdot$: $x \cdot y = y \cdot x$.
4. (a) $\cdot$ is distributive over $+$: $x \cdot (y + z) = (x \cdot y) + (x \cdot z)$.
   (b) $+$ is distributive over $\cdot$: $x + (y \cdot z) = (x + y) \cdot (x + z)$.
5. For every element $x \in B$, there exists an element $x' \in B$ (called the complement of x) such that: (a) $x + x' = 1$ and (b) $x \cdot x' = 0$.
6. There exists at least two elements $x, y \in B$ such that $x \neq y$.

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Duality

The Huntington postulates have been listed in pairs and designated by part (a) and part (b). One part may be obtained from the other if the binary operators and the identity elements are interchanged. This important property of Boolean algebra is called the *duality principle*. It states that every algebraic expression deducible from the postulates of Boolean algebra remains valid if the operators and identity elements are interchanged. In a two-valued Boolean algebra, the identity elements and the elements of the set B are the same: 1 and 0. The duality principle has many applications. If the *dual* of an algebraic expression is desired, we simply interchange OR and AND operators and replace 1's by 0's and 0's by 1's.

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Boolean Algebra vs Ordinary Algebra

1. Huntington postulates do not include the associative law. However, this law holds for Boolean algebra and can be derived (for both operators) from the other postulates.

$$x + (y + z) = (x + y) + z$$
$$x \,.\, (y \,.\, z) = (x \,.\, y) \,.\, z$$

2. The distributive law of + over ·, i.e., $x + (y \cdot z) = (x + y) \cdot (x + z)$, is valid for Boolean algebra, but not for ordinary algebra.

3. Boolean algebra does not have additive or multiplicative inverses; therefore, there are no subtraction or division operations.

4. Postulate 5 defines an operator called *complement* which is not available in ordinary algebra.

5. Ordinary algebra deals with the real numbers, which constitute an infinite set of elements. Boolean algebra deals with the as yet undefined set of elements B, but in the two-valued Boolean algebra defined below (and of interest in our subsequent use of this algebra), B is defined as a set with only two elements, 0 and 1.

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Postulates and Theorems of Boolean Algebra

Postulate 2	(a) $x + 0 = x$	(b) $x \cdot 1 = x$
Postulate 5	(a) $x + x' = 1$	(b) $x \cdot x' = 0$
Theorem 1	(a) $x + x = x$	(b) $x \cdot x = x$
Theorem 2	(a) $x + 1 = 1$	(b) $x \cdot 0 = 0$
Theorem 3, involution	$(x')' = x$	
Postulate 3, commutative	(a) $x + y = y + x$	(b) $xy = yx$
Theorem 4, associative	(a) $x + (y + z) = (x + y) + z$	(b) $x(yz) = (xy)z$
Postulate 4, distributive	(a) $x(y + z) = xy + xz$	(b) $x + yz = (x + y)(x + z)$
Theorem 5, DeMorgan	(a) $(x + y)' = x'y'$	(b) $(xy)' = x' + y'$
Theorem 6, absorption	(a) $x + xy = x$	(b) $x(x + y) = x$

THEOREM 1(a): $x + x = x$.

$x + x = (x + x) \cdot 1$	by postulate: 2(b)
$= (x + x)(x + x')$	5(a)
$= x + xx'$	4(b)
$= x + 0$	5(b)
$= x$	2(a)

THEOREM 1(b): $x \cdot x = x$.

$x \cdot x = xx + 0$	by postulate: 2(a)
$= xx + xx'$	5(b)
$= x(x + x')$	4(a)
$= x \cdot 1$	5(a)
$= x$	2(b)

Note that theorem 1(b) is the dual of theorem 1(a) and that each step of the proof in part (b) is the dual of part (a). Any dual theorem can be similarly derived from the proof of its corresponding pair.

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Theorems of Boolean Algebra (contd.)

THEOREM 2(a): $x + 1 = 1$.

$x + 1 = 1 \cdot (x + 1)$	by postulate: 2(b)
$= (x + x')(x + 1)$	5(a)
$= x + x' \cdot 1$	4(b)
$= x + x'$	2(b)
$= 1$	5(a)

THEOREM 2(b): $x \cdot 0 = 0$ by duality.

THEOREM 3: $(x')' = x$. From postulate 5, we have $x + x' = 1$ and $x \cdot x' = 0$, which defines the complement of x. The complement of x' is x and is also $(x')'$. Therefore, since the complement is unique, we have that $(x')' = x$.

THEOREM 6(a): $x + xy = x$.

$x + xy = x \cdot 1 + xy$	by postulate 2(b)
$= x(1 + y)$	by postulate 4(a)
$= x(y + 1)$	by postulate 3(a)
$= x \cdot 1$	by theorem 2(a)
$= x$	by postulate 2(b)

THEOREM 6(b): $x(x + y) = x$ by duality.

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Theorems of Boolean Algebra (contd.)

The algebraic proofs of the associative law and De Morgan's theorem are long and will not be shown here. However, their validity is easily shown with truth tables. For example, the truth table for the first De Morgan's theorem $(x + y)' = x'y'$ is shown below.

x	y	$x + y$	$(x + y)'$	x'	y'	$x'y'$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

Operator Precedence

The operator precedence for evaluating Boolean expressions is (1) parentheses, (2) NOT, (3) AND, and (4) OR.

Reference: M. Morris Mano, "Digital Logic and Computer Design", PHI

Examples

1. $x + x'y = (x + x')(x + y) = 1 \cdot (x + y) = x + y$

2. $x(x' + y) = xx' + xy = 0 + xy = xy$

3. $x'y'z + x'yz + xy' = x'z(y' + y) + xy' = x'z + xy'$

4. $$\begin{aligned} xy + x'z + yz &= xy + x'z + yz(x + x') \\ &= xy + x'z + xyz + x'yz \\ &= xy(1 + z) + x'z(1 + y) \\ &= xy + x'z \end{aligned}$$

5. $(x + y)(x' + z)(y + z) = (x + y)(x' + z)$ by duality from function 4.

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

COC2070 – Digital Logic and System Design

Prof. M. M. Sufyan Beg
Department of Computer Engineering
Z. H. College of Engineering & Technology
Aligarh Muslim University, India

More Examples

$$F_1' = (x'yz' + x'y'z)' = (x'yz')'(x'y'z)' = (x + y' + z)(x + y + z')$$

$$F_2' = [x(y'z' + yz)]' = x' + (y'z' + yz)' = x' + (y'z')' \cdot (yz)'$$

$$= x' + (y + z)(y' + z')$$

A simpler procedure for deriving the complement of a function is to take the dual of the function and complement each literal. This method follows from the generalized De Morgan's theorem. Remember that the dual of a function is obtained from the interchange of AND and OR operators and 1's and 0's.

Find the complement of the functions F_1 and F_2 of Example 2-2 by taking their duals and complementing each literal.

1. $F_1 = x'yz' + x'y'z$.
   The dual of F_1 is $(x' + y + z')(x' + y' + z)$.
   Complement each literal: $(x + y' + z)(x + y + z') = F_1'$.

2. $F_2 = x(y'z' + yz)$.
   The dual of F_2 is $x + (y' + z')(y + z)$.
   Complement each literal: $x' + (y + z)(y' + z') = F_2'$.

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Minterms and Maxterms

			Minterms		Maxterms	
x	y	z	Term	Designation	Term	Designation
0	0	0	$x'y'z'$	m_0	$x + y + z$	M_0
0	0	1	$x'y'z$	m_1	$x + y + z'$	M_1
0	1	0	$x'yz'$	m_2	$x + y' + z$	M_2
0	1	1	$x'yz$	m_3	$x + y' + z'$	M_3
1	0	0	$xy'z'$	m_4	$x' + y + z$	M_4
1	0	1	$xy'z$	m_5	$x' + y + z'$	M_5
1	1	0	xyz'	m_6	$x' + y' + z$	M_6
1	1	1	xyz	m_7	$x' + y' + z'$	M_7

Sum of Minterms – Canonical Sum of Product Form: $F(x, y, z) = \Sigma(1, 3, 6, 7)$

Product of Maxterms–Canonical Product of Sums: $F(x, y, z) = \Pi(0, 2, 4, 5)$

Sum of Product (SOP) Form $F_1 = y' + xy + x'yz'$

Product of Sums (POS) Form $F_2 = x(y' + z)(x' + y + z' + w)$

Non-standard form $F_3 = (AB + CD)(A'B' + C'D')$

Using the Distributive Law, may be changed to the standard form

$$F_3 = A'B'CD + ABC'D'$$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Other Logic Operations

2^{2^n} functions for n hinary variables.

Truth tables for the 16 functions of two binary variables

x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
Operator Symbol				/		/		$\oplus$	+	$\downarrow$	$\odot$	′	$\subset$	′	$\supset$	$\uparrow$	

Two functions that produce a constant 0 or 1.

Four functions with unary operations complement and transfer.

Ten functions with binary operators that define eight different operations AND, OR, NAND, NOR, exclusive-OR, equivalence, inhibition, and implication.

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

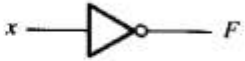
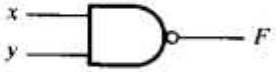
Other Logic Operations (contd.)

Boolean functions	Operator symbol	Name	Comments
$F_0 = 0$		Null	Binary constant 0
$F_1 = xy$	$x \cdot y$	AND	x and y
$F_2 = xy'$	x/y	Inhibition	x but not y
$F_3 = x$		Transfer	x
$F_4 = x'y$	y/x	Inhibition	y but not x
$F_5 = y$		Transfer	y
$F_6 = xy' + x'y$	$x \oplus y$	Exclusive-OR	x or y but not both
$F_7 = x + y$	$x + y$	OR	x or y
$F_8 = (x + y)'$	$x \downarrow y$	NOR	Not-OR
$F_9 = xy + x'y'$	$x \odot y$	Equivalence*	x equals y
$F_{10} = y'$	y'	Complement	Not y
$F_{11} = x + y'$	$x \subset y$	Implication	If y then x
$F_{12} = x'$	x'	Complement	Not x
$F_{13} = x' + y$	$x \supset y$	Implication	If x then y
$F_{14} = (xy)'$	$x \uparrow y$	NAND	Not-AND
$F_{15} = 1$		Identity	Binary constant 1

**Equivalence* is also known as *equality*, *coincidence*, and *exclusive-NOR*.

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Digital Logic Gates

AND		$F = xy$	x	y	F
			0	0	0
			0	1	0
			1	0	0
			1	1	1
OR		$F = x + y$	x	y	F
			0	0	0
			0	1	1
			1	0	1
			1	1	1
Inverter		$F = x'$	x	F	
			0	1	
			1	0	
Buffer		$F = x$	x	F	
			0	0	
			1	1	
NAND		$F = (xy)'$	x	y	F
			0	0	1
			0	1	1
			1	0	1
			1	1	0
NOR		$F = (x + y)'$	x	y	F
			0	0	1
			0	1	0
			1	0	0
			1	1	0
Exclusive-OR (XOR)		$F = xy' + x'y$ $= x \oplus y$	x	y	F
			0	0	0
			0	1	1
			1	0	1
			1	1	0
Exclusive-NOR or equivalence		$F = xy + x'y'$ $= x \odot y$	x	y	F
			0	0	1
			0	1	0
			1	0	0
			1	1	1

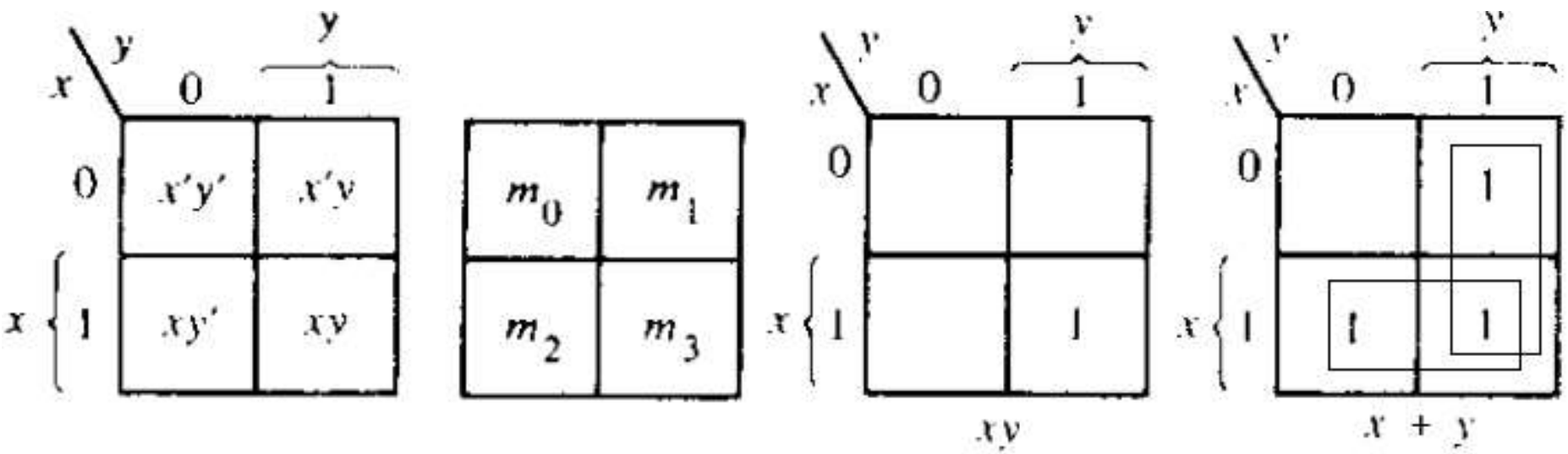
Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

COC2070 – Digital Logic and System Design

Prof. M. M. Sufyan Beg
Department of Computer Engineering
Z. H. College of Engineering & Technology
Aligarh Muslim University, India

Simplification of Boolean Functions

- Minimization to a minimum number of product (sum) terms in SOP(POS) form with each product(sum) term having the minimum number of variables in it
  - By Boolean Postulates and Theorems
  - By Karnaugh's Maps (or K-Maps), also called Veitch Diagram
- 2 variable K-Map



Reference: M. Morris Mano, "Digital Logic and Computer Design", PHI

3 – Variable K-Map

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6

$x \backslash yz$	00	01	11	10
0	$x'y'z'$	$x'y'z$	$x'yz$	$x'yz'$
1	$xy'z'$	$xy'z$	xyz	xyz'

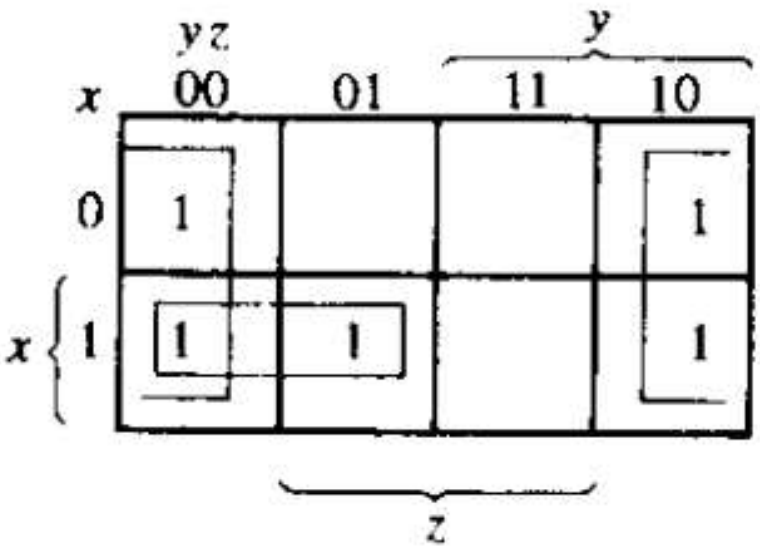
$x \backslash yz$	00	01	11	10
0			1	1
1	1	1		

$$x'yz + x'yz' + xy'z' + xy'z = m_3 + m_2 + m_4 + m_5$$

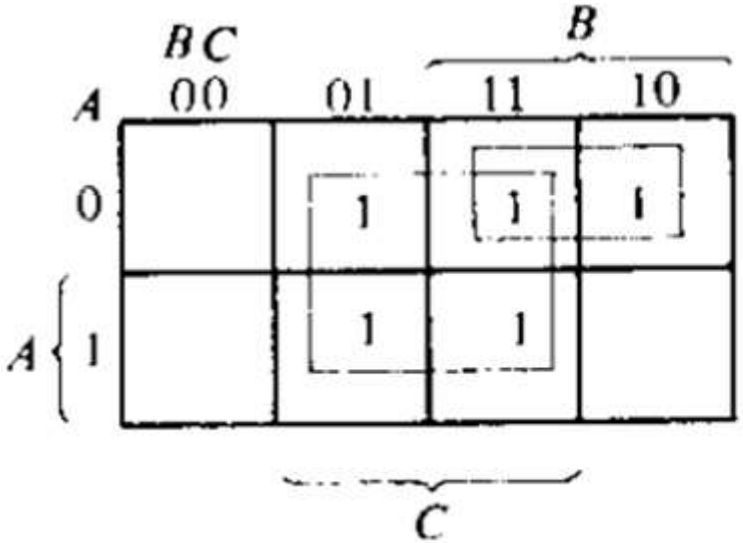
$$F(x,y,z) = \Sigma(2,3,4,5) = x'y + xy'$$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

3 – Variable K-Map (contd.)



$$f(x, y, z) = \Sigma(0, 2, 4, 5, 6) = z' + xy'$$



$A'C + A'B + AB'C + BC$

$= A'C(B+B') + A'B(C+C') + AB'C + (A+A')BC$

$= A'BC + A'B'C + \cancel{A'BC} + A'BC' + AB'C + ABC + \cancel{A'BC}$

$= A'B'C + A'BC' + A'BC + AB'C + ABC$

$F(A,B,C) = m_1 + m_2 + m_3 + m_5 + m_7 = \Sigma(1,2,3,5,7) = C + A'B$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

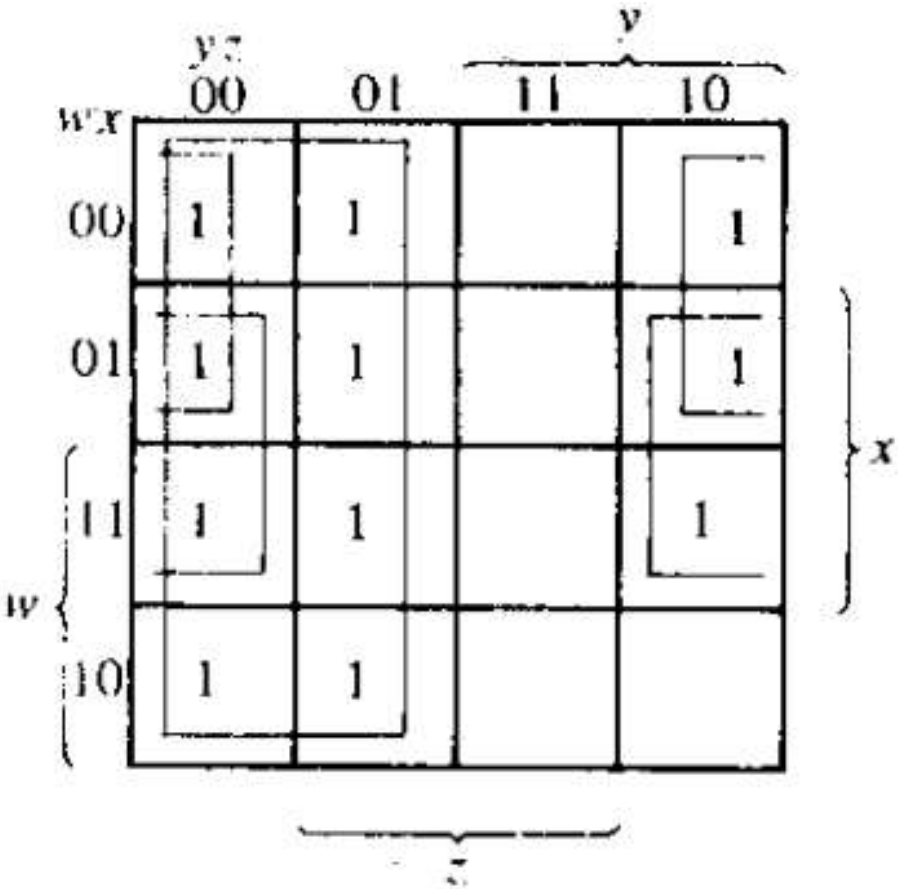
4 – Variable K-Map

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6
m_{12}	m_{13}	m_{15}	m_{14}
m_8	m_9	m_{11}	m_{10}

$wx \backslash yz$	00	01	11	10
00	$w'x'y'z'$	$w'x'y'z$	$w'x'yz$	$w'x'yz'$
01	$w'xy'z'$	$w'xy'z$	$w'xyz$	$w'xyz'$
11	$wxy'z'$	$wxy'z$	$wxyz$	$wxyz'$
10	$wx'y'z'$	$wx'y'z$	$wx'yz$	$wx'yz'$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

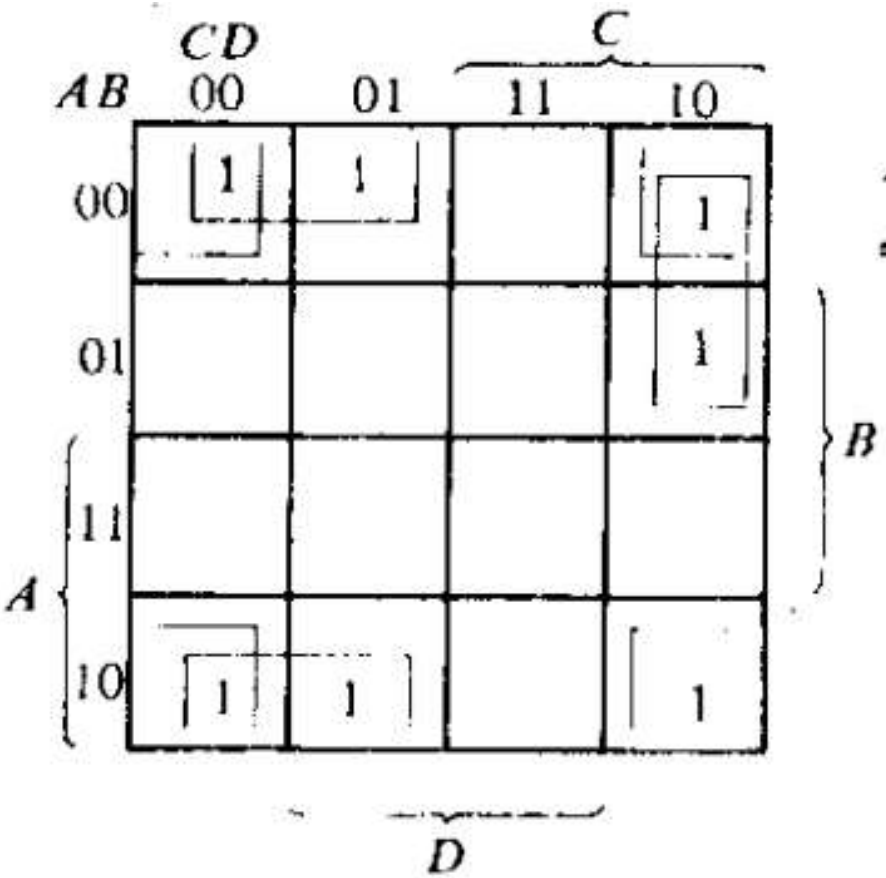
4 – Variable K-Map (contd.)



$$F(w, x, y, z)$$
$$= \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$$
$$= y' + w'z' + xz'$$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

4 – Variable K-Map (contd.)



$$A'B'C' + B'CD' + A'BCD' + AB'C'$$
$$= B'D' + B'C' + A'CD'$$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

COC2070 – Digital Logic and System Design

Prof. M. M. Sufyan Beg
Department of Computer Engineering
Z. H. College of Engineering & Technology
Aligarh Muslim University, India

5 – Variable K-Map (contd.)

AB \ CDE	000	001	011	010	110	111	101	100
00	0	1	3	2	6	7	5	4
01	8	9	11	10	14	15	13	12
11	24	25	27	26	30	31	29	28
10	16	17	19	18	22	23	21	20

C

B

A

D

E

E

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

6 – Variable K-Map (contd.)

ABC \ DEF	000	001	011	010	110	111	101	100
000	0	1	3	2	6	7	5	4
001	8	9	11	10	14	15	13	12
011	24	25	27	26	30	31	29	28
010	16	17	19	18	22	23	21	20
110	48	49	51	50	54	55	53	52
111	56	57	59	58	62	63	61	60
101	40	41	43	42	46	47	45	44
100	32	33	35	34	38	39	37	36

D

C

B

C

A

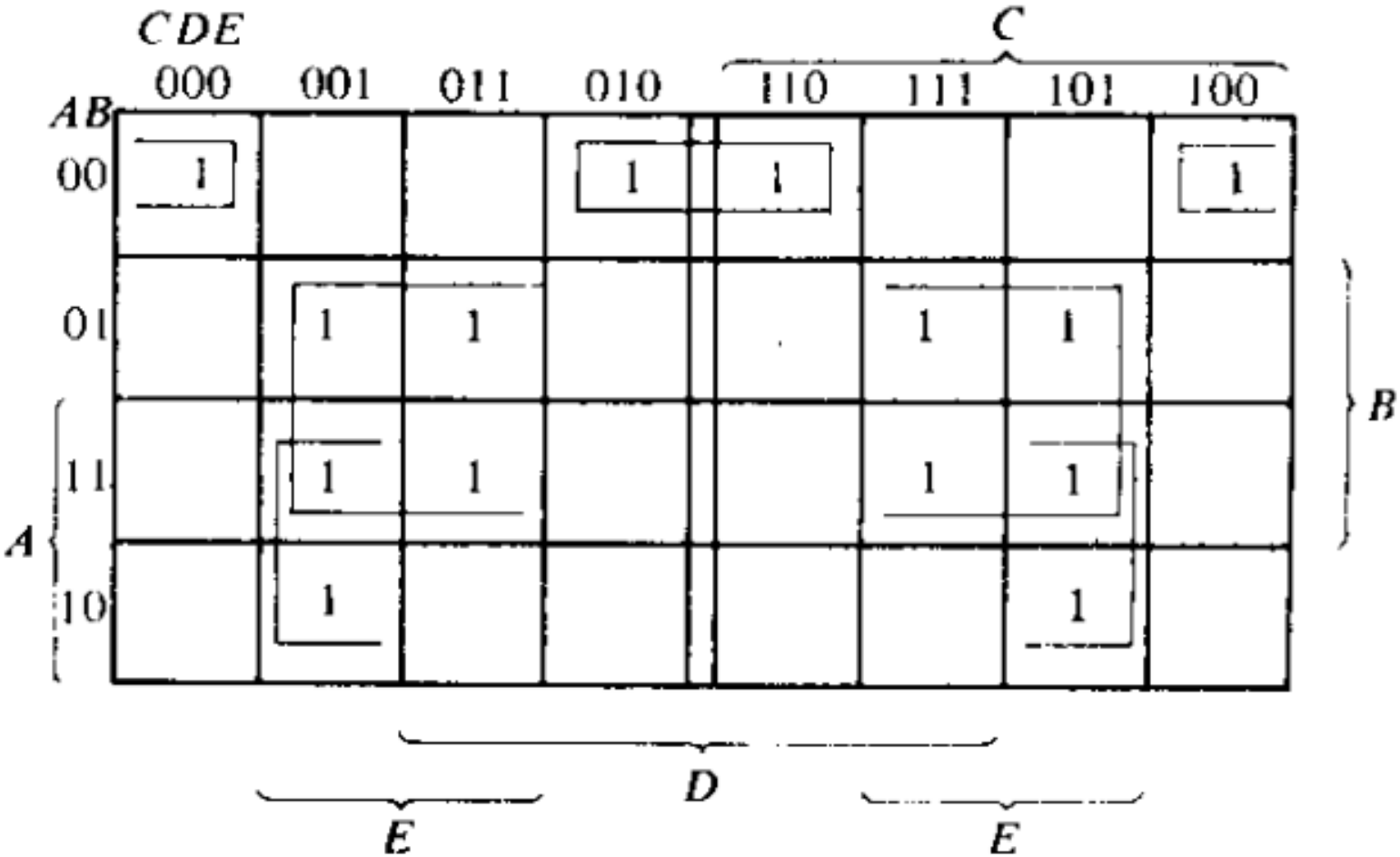
E

F

F

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Example of 5 – Variable K-Map



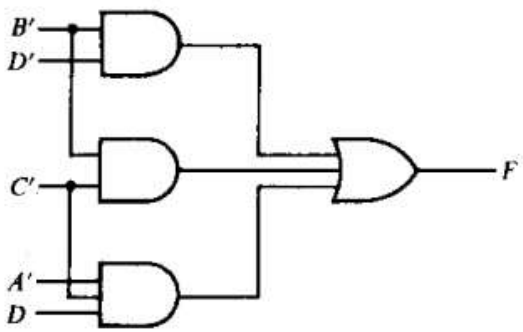
$$F(A, B, C, D, E)$$
$$= \Sigma(0, 2, 4, 6, 9, 11, 13, 15, 17, 21, 25, 27, 29, 31)$$
$$= BE + AD'E + A'B'E'$$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

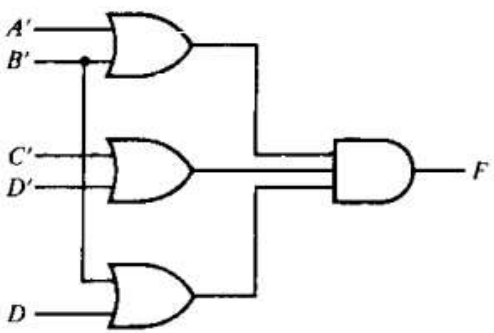
POS Simplification

AB \ CD	00	01	11	10
00	1	1	0	1
01	0	1	0	0
11	0	0	0	0
10	1	1	0	1

$$F(A, B, C, D)$$
$$= \Sigma(0, 1, 2, 5, 8, 9, 10)$$
$$= B'D' + B'C' + A'C'D$$
$$= (A' + B')(C' + D')(B' + D)$$



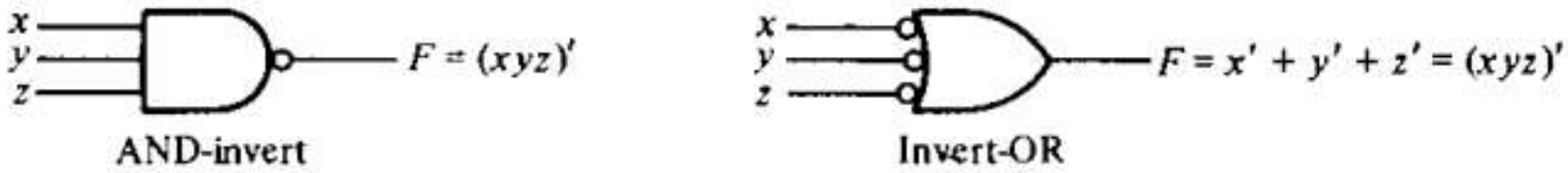
$F = B'D' + B'C' + A'C'D$



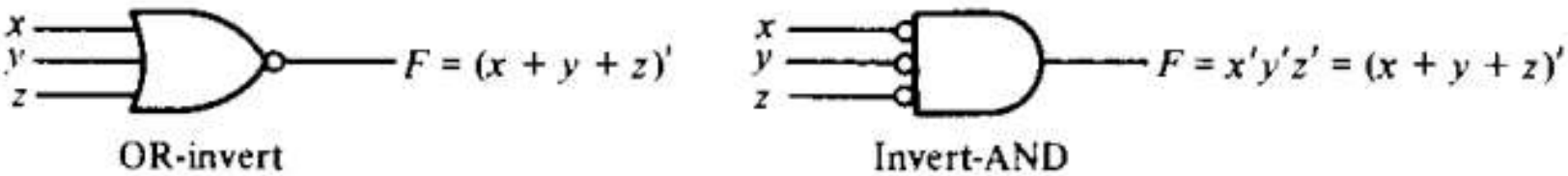
$F = (A' + B')(C' + D')(B' + D)$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

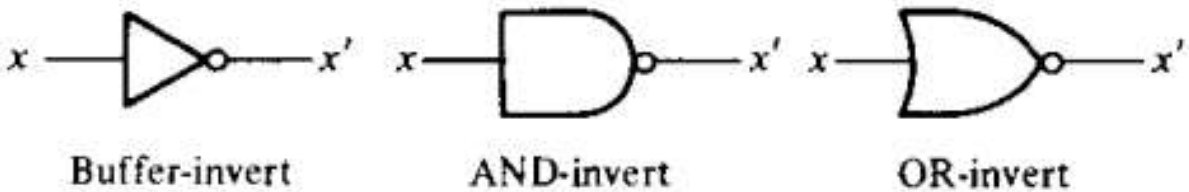
Graphic Symbols for NAND and NOR Gates



(a) Two graphic symbols for NAND gate.



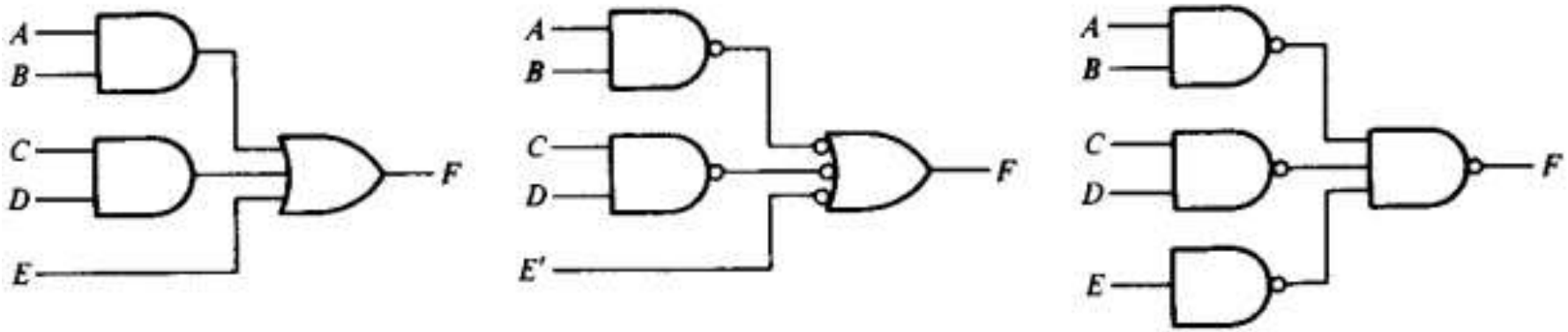
(b) Two graphic symbols for NOR gate.



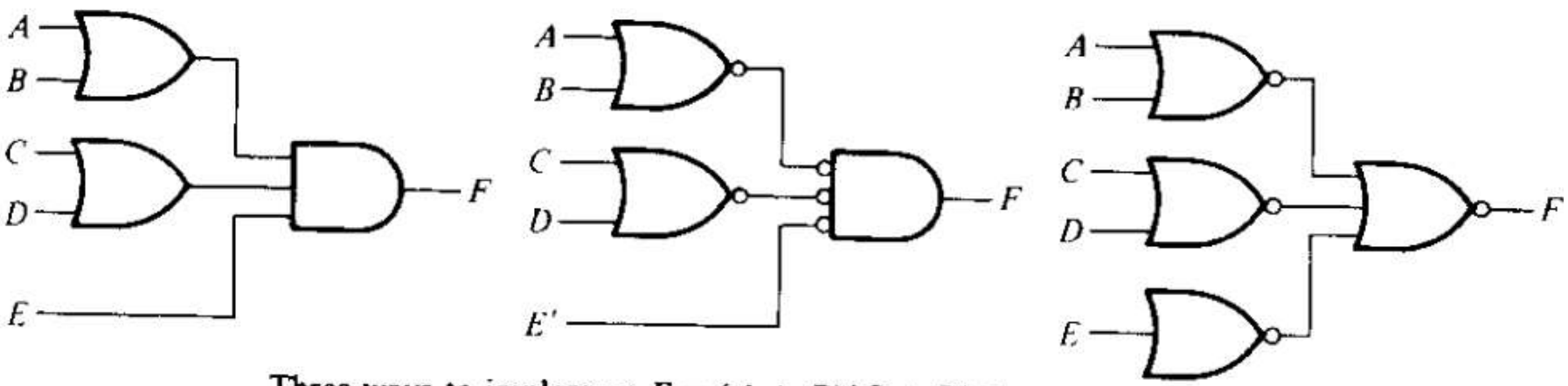
(c) Three graphic symbols for inverter.

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

NAND-NAND/NOR-NOR Implementation



Three Ways to Implement $F = AB + CD + E$



Three ways to implement $F = (A + B)(C + D)E$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

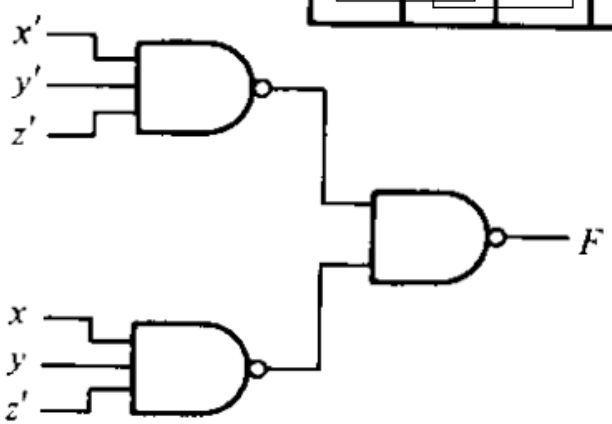
Rules for NAND and NOR Implementation

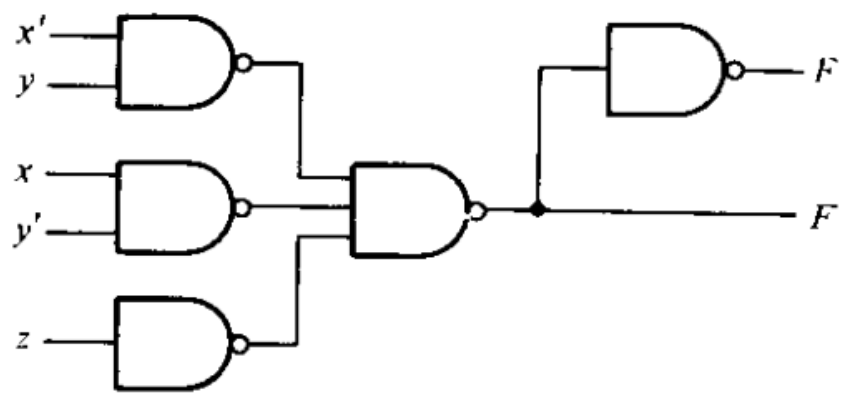
Case	Function to simplify	Standard form to use	How to derive	Implement with	Number of levels to F
(a)	F	Sum of products	Combine 1's in map	NAND	2
(b)	F'	Sum of products	Combine 0's in map	NAND	3
(c)	F	Product of sums	Complement F' in (b)	NOR	2
(d)	F'	Product of sums	Complement F in (a)	NOR	3

x \ yz	00	01	11	10
0	1	0	0	0
	0	0	0	1

$$F = x'y'z' + xyz'$$

$$F' = x'y + xy' + z$$





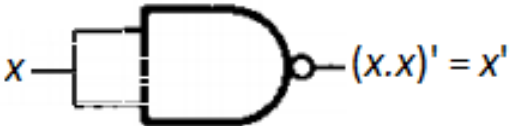
Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

COC2070 – Digital Logic and System Design

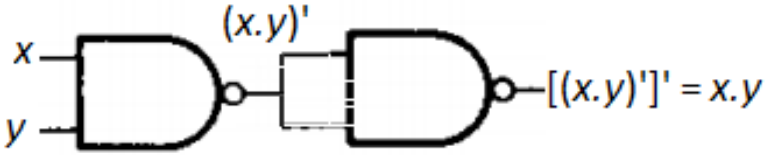
Prof. M. M. Sufyan Beg
Department of Computer Engineering
Z. H. College of Engineering & Technology
Aligarh Muslim University, India

NAND and NOR as Universal/Functionally Complete Gates

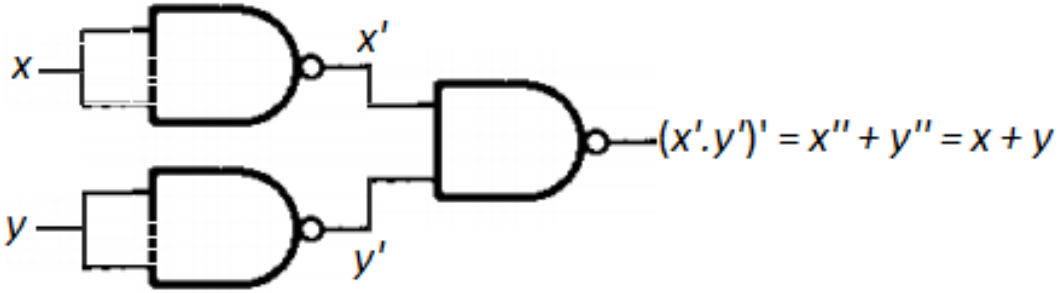
- NOT using NAND only



- AND using NAND only



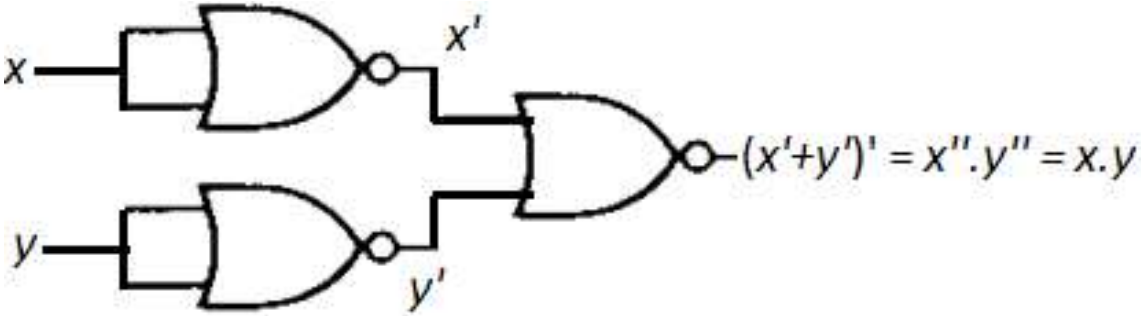
- OR using NAND only



- NOT using NOR only

- OR using NOR only

- AND using NOR only



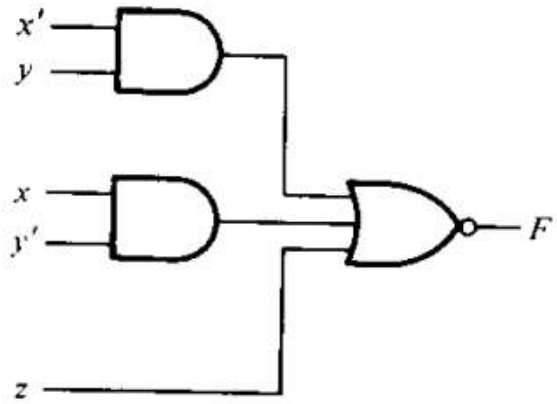
2-Level Implementations

- We consider 4 types of Gates: AND, OR, NAND, NOR
- 16 combinations are possible for 2-Levels
- 8 of these are degenerate forms as they collapse into a single operation
- 8 non-degenerate forms are:
  - AND-OR OR-AND
  - NAND-NAND NOR-NOR
  - NOR-OR NAND-AND
  - OR-NAND AND-NOR
  - Note: the two forms in the same line are the dual of each other

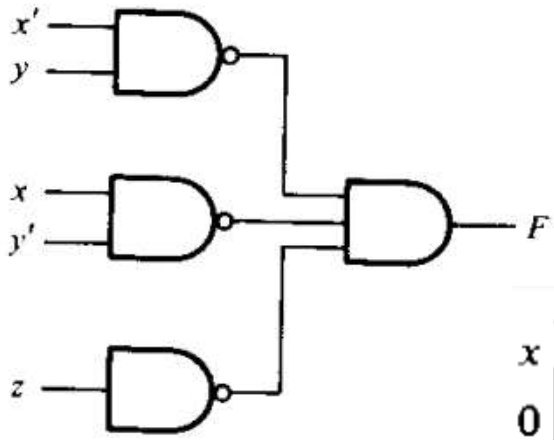
Equivalent nondegenerate form (a)	Equivalent nondegenerate form (b)	Implements the function	Simplify F' in	To get an output of
AND-NOR	NAND-AND	AND-OR-INVERT	Sum of products by combining 0's in the map	F
OR-NAND	NOR-OR	OR-AND-INVERT	Product of sums by combining 1's in the map and then complementing	F

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

NAND-NAND/NOR-NOR Implementation



AND-NOR



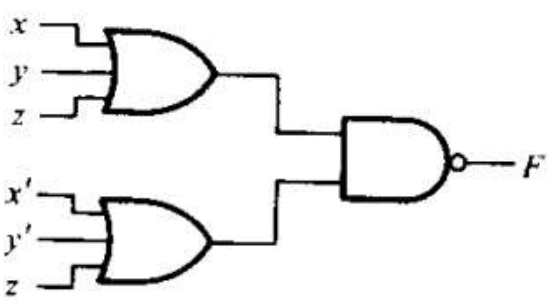
NAND-AND

(a) $F = (x'y + xy' + z)'$

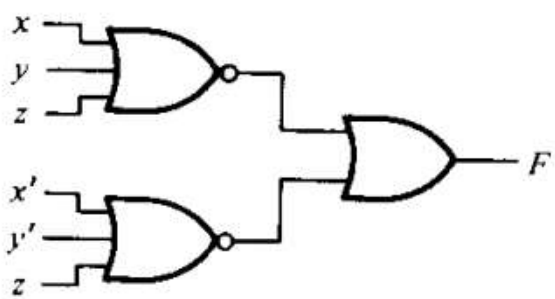
x \ yz	00	01	11	10
0	1	0	0	0
	0	0	0	1

$F = x'y'z' + xyz'$

$F' = x'y + xy' + z$



OR-NAND



NOR-OR

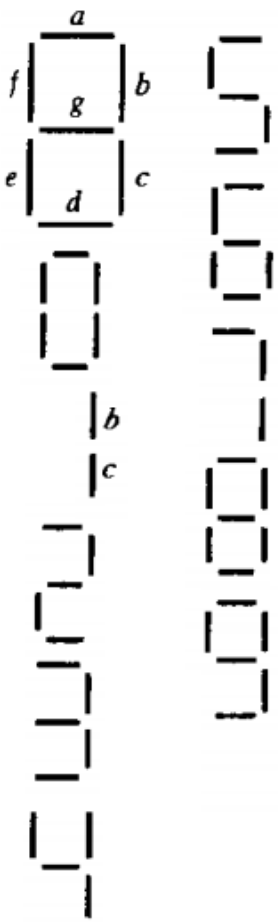
(b) $F = [(x + y + z)(x' + y' + z)]'$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Don’t Care Condition

- Where certain combinations of input variables never occur
- We don’t care what the output is to be for these combinations
- Can be used to provide further simplification
- e.g. Design of BCD to 7-Segment Decoder

A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1



Reference: https://www.geeksforgeeks.org/bcd-to-7-segment-decoder/

Design of BCD to 7-Segment Decoder

for a:

AB\CD	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	X	X	X	X
10	1	1	X	X

F(ABCD)= ¬B¬D + C + BD + A

for b:

AB\CD	00	01	11	10
00	1	1	1	1
01	1	0	1	0
11	X	X	X	X
10	1	1	X	X

F(ABCD)= ¬B + ¬C¬D + CD

for c:

AB\CD	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	X	X	X	X
10	1	1	X	X

F(ABCD)= ¬C + D + B

for d:

AB\CD	00	01	11	10
00	1	0	1	1
01	0	1	0	1
11	X	X	X	X
10	1	1	X	X

F(ABCD)=¬B¬D+¬BC+B¬CD+C¬D+A

for e:

AB\CD	00	01	11	10
00	1	0	0	1
01	0	0	0	1
11	X	X	X	X
10	1	0	X	X

F(ABCD)= ¬B¬D + C¬D

for f:

AB\CD	00	01	11	10
00	1	0	0	0
01	1	1	0	1
11	X	X	X	X
10	1	1	X	X

F(ABCD)=¬C¬D + B¬C + B¬D + A

for g:

AB\CD	00	01	11	10
00	0	0	1	1
01	1	1	0	1
11	X	X	X	X
10	1	1	X	X

F(ABCD)= ¬BC + B¬C + A + B¬D

Reference: https://www.geeksforgeeks.org/bcd-to-7-segment-decoder/

Variations of K-Maps

Variations of the three-variable map

$z \backslash xy$	00	01	11	10
0	0	2	6	4
1	1	3	7	5

yz	0	1
00	0	4
01	1	5
11	3	7
10	2	6

Variations of the four-variable map

$CD \backslash AB$	00	01	11	10
00	0	4	12	8
01	1	5	13	9
11	3	7	15	11
10	2	6	14	10

12	14	6	4
13	15	7	5
9	11	3	1
8	10	2	0

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

COC2070 – Digital Logic and System Design

Prof. M. M. Sufyan Beg
Department of Computer Engineering
Z. H. College of Engineering & Technology
Aligarh Muslim University, India

Tabulation Method (Quine-McKlusky Method)

$$F = \Sigma(0, 1, 2, 8, 10, 11, 14, 15)$$

(a)

	w x y z	
0	0 0 0 0	√
1	0 0 0 1	√
2	0 0 1 0	√
8	1 0 0 0	√
10	1 0 1 0	√
11	1 0 1 1	√
14	1 1 1 0	√
15	1 1 1 1	√

(b)

	w x y z	
0, 1	0 0 0 –	
0, 2	0 0 – 0	√
0, 8	– 0 0 0	√
2, 10	– 0 1 0	√
8, 10	1 0 – 0	√
10, 11	1 0 1 –	√
10, 14	1 – 1 0	√
11, 15	1 – 1 1	√
14, 15	1 1 1 –	√

(c)

	w x y z
0, 2, 8, 10	– 0 – 0
0, 8, 2, 10	– 0 – 0
10, 11, 14, 15	1 – 1 –
10, 14, 11, 15	1 – 1 –

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Tabulation Method $F(w, x, y, z) = \Sigma(1, 4, 6, 7, 8, 9, 10, 11, 15)$

(a)			(b)			wxyz	(c)	wxyz
0001	1	√	1, 9	(8)		-001	8, 9, 10, 11	(1, 2)10--
0100	4	√	4, 6	(2)		01-0	~~8, 9, 10, 11~~	~~(1, 2)10--~~
1000	8	√	8, 9	(1)	√	100-		
			8, 10	(2)	√	10-0		
0110	6	√						
1001	9	√	6, 7	(1)		011-		
1010	10	√	9, 11	(2)	√	10-1		
			10, 11	(1)	√	101-		
0111	7	√						
1011	11	√	7, 15	(8)		-111		
			11, 15	(4)		1-11		
1111	15	√						

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Tabulation Method

$$F = x'y'z + w'xz' + wx' + xyz$$

Prime-implicants

Decimal	Binary $w\ x\ y\ z$	Term
1, 9 (8)	$-\ 0\ 0\ 1$	$x'y'z$
4, 6 (2)	$0\ 1\ -\ 0$	$w'xz'$
6, 7 (1)	$0\ 1\ 1\ -$	$w'xy$
7, 15 (8)	$-\ 1\ 1\ 1$	xyz
11, 15 (4)	$1\ -\ 1\ 1$	wyz
8, 9, 10, 11 (1, 2)	$1\ 0\ -\ -$	wx'

		1	4	6	7	8	9	10	11	15
$\surd\ x'y'z$	1, 9	X					X			
$\surd\ w'xz'$	4, 6		X	X						
$w'xy$	6, 7			X	X					
xyz	7, 15				X					X
wyz	11, 15								X	X
$\surd\ wx'$	8, 9, 10, 11					X	X	X	X	
		$\surd$	$\surd$	$\surd$		$\surd$	$\surd$	$\surd$	$\surd$	

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Unit 1 Completed

BOOLEAN ALGEBRA AND LOGIC GATES

Introduction, Binary numbers, Base-conversions, Octal and hexadecimal numbers, complements, binary codes, Basic Theorems and properties, Boolean functions and representation in canonical and standard forms, SOP and POS forms, other logic operations, Digital logic gates, Function minimization: Karnaugh map methods, Limitations of K-maps for larger variables, POS-simplification, NAND/NOR implementation, other 2-level implementations, Don't-care conditions, Tabular method.

Course Outcome

1. Understand Boolean logic operation, minimization of a Boolean function and its implementation.